

1. Hello World C++ Code

```
#include <iostream>
using namespace std;

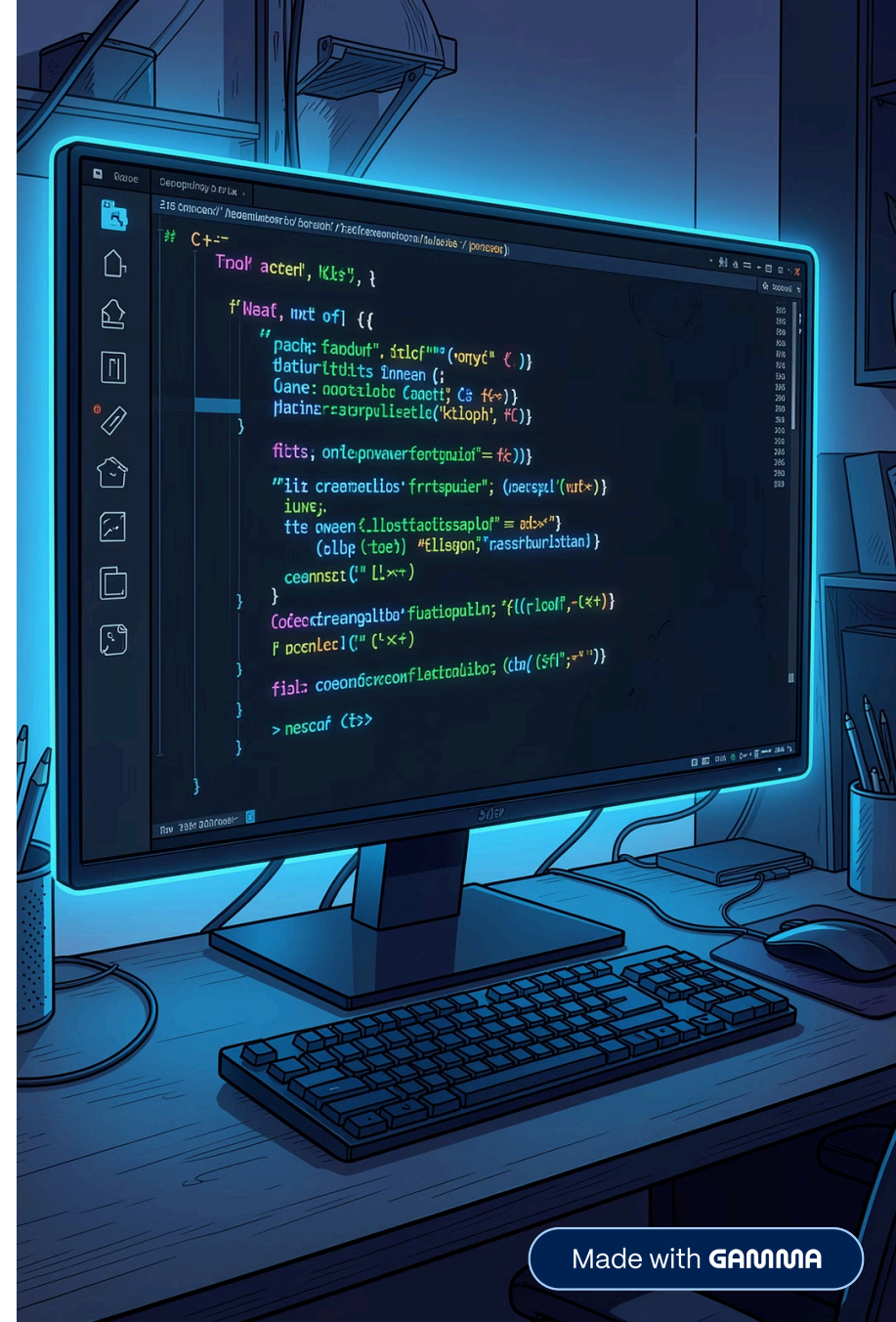
int main() {
    cout << "Hello World";
    return 0;
}
```

Sample Input

No Input

Sample Output

Hello World



2. Input and Output Using cin, cout

C++ Code

```
#include <iostream>
using namespace std;

int main() {
    int num;
    cout << "Enter Number: ";
    cin >> num;
    cout << "Number = " << num;
    return 0;
}
```

Sample Input

25

Sample Output

Enter Number: 25
Number = 25

3. Swap Two Numbers Using Temporary Variable

C++ Code

```
#include <iostream>
using namespace std;

int main() {
    int a, b, temp;
    cin >> a >> b;

    temp = a;
    a = b;
    b = temp;
    cout << "a = " << a << endl;
    cout << "b = " << b;
    return 0;
}
```

Sample Input

10 20

Sample Output

a = 20
b = 10

4. Swap Without Temporary Variable

C++ Code

```
#include <iostream>
using namespace std;

int main() {
    int a, b;
    cin >> a >> b;

    a = a + b;
    b = a - b;
    a = a - b;
    cout << a << " " << b;
    return 0;
}
```

Sample Input

15 30

Sample Output

30 15

5. Prime Number

C++ Code

```
#include <iostream>
using namespace std;

int main() {
    int n, i;
    cin >> n;

    for(i=2; i<n; i++)
        if(n%i==0) break;
    if(i==n)
        cout<<"Prime";
    else
        cout<<"Not Prime";
    return 0;
}
```

Sample Input

17

Sample Output

Prime

6. Factorial Using Loop

C++ Code

```
#include <iostream>
using namespace std;

int main() {
    int n;
    long long fact=1;
    cin>>n;

    for(int i=1;i<=n;i++)
        fact*=i;
    cout<<fact;
    return 0;
}
```

Sample Input

5

Sample Output

120

7. Even or Odd

C++ Code

```
#include <iostream>
using namespace std;

int main(){
    int n;
    cin>>n;

    if(n%2==0)
        cout<<"Even";
    else
        cout<<"Odd";
    return 0;
}
```

Sample Input

12

Sample Output

Even

8. Leap Year

C++ Code

```
#include <iostream>
using namespace std;

int main(){
    int year;
    cin>>year;

    if((year%400==0) || ((year%4==0)&&(year%100!=0)))
        cout<<"Leap Year";
    else
        cout<<"Not Leap Year";
    return 0;
}
```

Sample Input

2024

Sample Output

Leap Year

9. Simple Calculator

C++ Code

```
#include <iostream>
using namespace std;

int main(){
    int a,b;
    cin>>a>>b;

    cout<<"Addition = "<<a+b<<endl;
    cout<<"Subtraction = "<<a-b<<endl;
    cout<<"Multiplication = "<<a*b<<endl;
    cout<<"Division = "<<a/b;
    return 0;
}
```

Sample Input

20 5

Sample Output

Addition = 25
Subtraction = 15
Multiplication = 100
Division = 4

10. Reverse a Number

C++ Code

```
#include <iostream>
using namespace std;

int main(){
    int n,rev=0;

    cin>>n;
    while(n>0){
        rev=rev*10+n%10;
        n/=10;
    }
    cout<<rev;
    return 0;
}
```

Sample Input

12345

Sample Output

54321

11. Palindrome Number

C++ Code

```
#include <iostream>
using namespace std;

int main(){
    int n,temp,rev=0;

    cin>>n;
    temp=n;
    while(n>0){
        rev=rev*10+n%10;
        n/=10;
    }
    if(temp==rev)
        cout<<"Palindrome";
    else
        cout<<"Not Palindrome";
    return 0;
}
```

Sample Input

121

Sample Output

Palindrome

12. Factorial Using Recursion

C++ Code

```
#include <iostream>
using namespace std;

int fact(int n){
    if(n==0) return 1;
    return n*fact(n-1);
}

int main(){
    int n;
    cin>>n;

    cout<<fact(n);
    return 0;
}
```

Sample Input

6

Sample Output

720

13. Fibonacci Series

C++ Code

```
#include <iostream>
using namespace std;

int main(){
    int n,a=0,b=1,c;

    cin>>n;
    cout<<a<<" "<<b<<" ";
    for(int i=3;i<=n;i++){
        c=a+b;
        cout<<c<<" ";
        a=b;
        b=c;
    }
    return 0;
}
```

Sample Input

7

Sample Output

0 1 1 2 3 5 8

14. GCD

C++ Code

```
#include <iostream>
using namespace std;

int main(){
    int a,b;

    cin>>a>>b;
    while(b!=0){
        int t=b;
        b=a%b;
        a=t;
    }
    cout<<a;
    return 0;
}
```

Sample Input

24 36

Sample Output

12

15. LCM

C++ Code

```
#include <iostream>
using namespace std;

int main(){
    int a,b,x,y;

    cin>>a>>b;
    x=a;
    y=b;
    while(y!=0){
        int t=y;
        y=x%y;
        x=t;
    }
    cout<<(a*b)/x;
    return 0;
}
```

Sample Input

12 18

Sample Output

36

Class and Object

C++ Code

```
#include <iostream>
using namespace std;

class Student {
public:
    int id;
    string name;

    void getData() {
        cin >> id >> name;
    }
    void display() {
        cout << "ID : " << id << endl;
        cout << "Name : " << name;
    }
};

int main() {
    Student s;
    s.getData();
    s.display();
    return 0;
}
```

Sample Input

101 Rahul

Sample Output

ID : 101

Name : Rahul

17. Constructor

C++ Code

```
#include <iostream>
using namespace std;

class Student {
public:
    Student() {
        cout << "Constructor Called";
    }
};

int main() {
    Student s;
    return 0;
}
```

Sample Input

No Input

Sample Output

Constructor Called

18. Destructor

C++ Code

```
#include <iostream>
using namespace std;

class Demo {
public:
    Demo() {
        cout << "Constructor Called\n";
    }

    ~Demo() {
        cout << "Destructor Called";
    }
};

int main() {
    Demo d;
    return 0;
}
```

Sample Output

Constructor Called
Destructor Called

19. Access Modifiers

C++ Code

```
#include <iostream>
using namespace std;

class Student {
private:
    int age;

public:
    void setAge(int a) {
        age = a;
    }

    void show() {
        cout << "Age = " << age;
    }
};

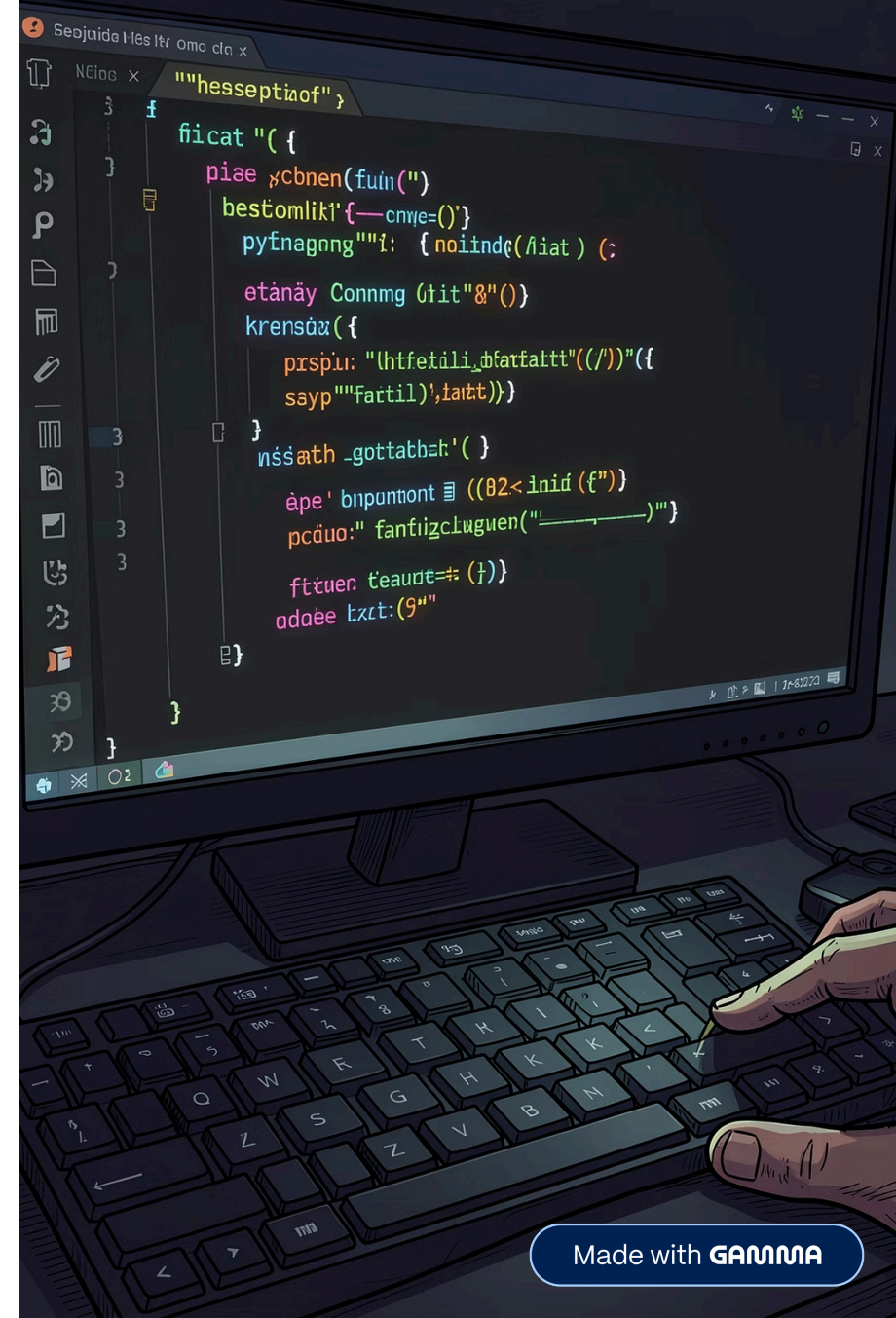
int main() {
    Student s;
    s.setAge(20);
    s.show();
    return 0;
}
```

Sample Output

Age = 20

C++ Object-Oriented Programming

Code examples covering Encapsulation, Data Abstraction, Inline Functions, Static Members, Friend Functions, This Pointer, Const Member Functions, and Inheritance types.



Encapsulation & Data Abstraction

20. Encapsulation

C++ Code

```
#include <iostream>
using namespace std;

class Employee {
private:
    int salary;

public:
    void setSalary(int s) { salary = s; }
    int getSalary() { return salary; }
};

int main() {
    Employee e;
    e.setSalary(50000);
    cout << e.getSalary();
}
```

Sample Output

50000

21. Data Abstraction

C++ Code

```
#include <iostream>
using namespace std;

class Car {
public:
    void start() {
        cout << "Car Started";
    }
};

int main() {
    Car c;
    c.start();
}
```

Sample Output

Car Started

Inline Function & Static Data Member

22. Inline Function

C++ Code

```
#include <iostream>
using namespace std;

class Demo {
public:
    inline int square(int x) {
        return x * x;
    }
};

int main() {
    Demo d;
    cout << d.square(6);
}
```

Sample Input

6

Sample Output

36

23. Static Data Member

C++ Code

```
#include <iostream>
using namespace std;

class Demo {
public:
    static int count;
    Demo() { count++; }
};

int Demo::count = 0;

int main() {
    Demo a, b, c;
    cout << Demo::count;
}
```

Sample Output

3

Static Member Function & Friend Function

24. Static Member Function

C++ Code

```
#include <iostream>
using namespace std;

class Demo {
public:
    static void show() {
        cout << "Static Function";
    }
};

int main() {
    Demo::show();
}
```

Sample Output

Static Function

25. Friend Function

C++ Code

```
#include <iostream>
using namespace std;

class Demo {
private:
    int num = 50;
    friend void display(Demo);
};

void display(Demo d) {
    cout << d.num;
}

int main() {
    Demo d;
    display(d);
}
```

Sample Output

50

This Pointer & Const Member Function

26. This Pointer

C++ Code

```
#include <iostream>
using namespace std;

class Demo {
    int x;
public:
    void setData(int x) {
        this->x = x;
    }
    void display() {
        cout << x;
    }
};

int main() {
    Demo d;
    d.setData(100);
    d.display();
}
```

Sample Output

100

27. Const Member Function

C++ Code

```
#include <iostream>
using namespace std;

class Demo {
public:
    void show() const {
        cout << "Constant Member Function";
    }
};

int main() {
    Demo d;
    d.show();
}
```

Sample Output

Constant Member Function

Single, Multiple & Multilevel Inheritance

28. Single Inheritance

C++ Code

```
#include <iostream>
using namespace std;

class Animal {
public:
    void sound() {
        cout << "Animal Sound\n";
    }
};

class Dog : public Animal {
public:
    void bark() {
        cout << "Dog Barks";
    }
};

int main() {
    Dog d;
    d.sound();
    d.bark();
}
```

Sample Output

Animal Sound
Dog Barks

29. Multiple Inheritance

C++ Code

```
#include <iostream>
using namespace std;

class A {
public:
    void showA() {
        cout << "Class A\n";
    }
};

class B {
public:
    void showB() {
        cout << "Class B\n";
    }
};

class C : public A, public B {};

int main() {
    C obj;
    obj.showA();
    obj.showB();
}
```

Sample Output

Class A
Class B

30. Multilevel Inheritance

C++ Code

```
#include <iostream>
using namespace std;

class A {
public:
    void displayA() {
        cout << "Class A\n";
    }
};

class B : public A {
public:
    void displayB() {
        cout << "Class B\n";
    }
};

class C : public B {
public:
    void displayC() {
        cout << "Class C";
    }
};

int main() {
    C obj;
    obj.displayA();
    obj.displayB();
    obj.displayC();
}
```

Sample Output

Class A
Class B
Class C

Encapsulation Example (Bundle Data and Functions)

C++ Code

```
#include <iostream>
using namespace std;

class Employee {
private:
    int id;
    string name;
    float salary;
public:
    void setData(int i, string n, float s) {
        id = i;
        name = n;
        salary = s;
    }
    void displayData() {
        cout << "Employee ID : " << id << endl;
        cout << "Employee Name : " << name << endl;
        cout << "Employee Salary: " << salary << endl;
    }
};

int main() {
    Employee emp;
    int id;
    string name;
    float salary;
    cout << "Enter Employee ID: ";
    cin >> id;
    cout << "Enter Employee Name: ";
    cin >> name;
    cout << "Enter Employee Salary: ";
    cin >> salary;
    emp.setData(id, name, salary);
    cout << "\nEmployee Details\n";
    emp.displayData();
    return 0;
}
```

Sample Input

```
Enter Employee ID: 101
Enter Employee Name: Rahul
Enter Employee Salary: 45000
```

Sample Output

```
Employee Details
Employee ID : 101
Employee Name : Rahul
Employee Salary: 45000
```

Data Abstraction (Expose Essential Features Only)

C++ Code

```
#include <iostream>
using namespace std;

class ATM {
private:
    int balance;
public:
    ATM() {
        balance = 10000; // Initial Balance
    }
    void withdraw(int amount) {
        if (amount <= balance) {
            balance = balance - amount;
            cout << "Withdrawal Successful!" << endl;
        } else {
            cout << "Insufficient Balance!" << endl;
        }
    }
    void showBalance() {
        cout << "Available Balance: " << balance << endl;
    }
};

int main() {
    ATM user;
    int amount;
    cout << "Enter Amount to Withdraw: ";
    cin >> amount;
    user.withdraw(amount);
    user.showBalance();
    return 0;
}
```

Sample Input

Enter Amount to Withdraw: 2500

Sample Output

Withdrawal Successful!
Available Balance: 7500

Inline Functions in Class (Declare and Define Inline Functions Inside a Class)

C++ Code

```
#include <iostream>
using namespace std;

class Square {
public:
    // Inline function declared and defined inside the class
    inline int findSquare(int num) {
        return num * num;
    }
};

int main() {
    Square obj;
    int n;
    cout << "Enter a Number: ";
    cin >> n;
    cout << "Square of " << n << " = " << obj.findSquare(n);
    return 0;
}
```

Sample Input

Enter a Number: 7

Sample Output

Square of 7 = 49

Static Data Members & Static Member Functions (Use Shared Variables Across Objects)

C++ Code

```
#include <iostream>
using namespace std;

class Student {
private:
    int rollNo;
    static int totalStudents; // Static Data Member
public:
    Student() { totalStudents++; }
    void setRollNo(int r) { rollNo = r; }
    void display() {
        cout << "Roll Number : " << rollNo << endl;
    }
    // Static Member Function
    static void showTotalStudents() {
        cout << "Total Students Created : " << totalStudents << endl;
    }
};

// Definition of Static Data Member
int Student::totalStudents = 0;

int main() {
    Student s1, s2, s3;
    s1.setRollNo(101);
    s2.setRollNo(102);
    s3.setRollNo(103);
    s1.display();
    s2.display();
    s3.display();
    Student::showTotalStudents();
    return 0;
}
```

Sample Input

No Input

Sample Output

Roll Number : 101
Roll Number : 102
Roll Number : 103
Total Students Created : 3